- **介绍**
  - npm（node package manage）node 包 管理器。管理node包的工具。
  - 包是什么？包就是模块。（包约等于模块，一个包可以包括一个或多个模块）
  - npm这个工具，在安装 node 的时候，就已经安装到你的计算机中了。
  - 命令行中执行：`npm -v`，如果看到版本号，说明安装成功了。

- **什么是第三方模块**
  - 非node自带的模块。也不是自定义的模块。
  - 是别人写的模块，然后发布到npm网站，我们可以使用npm工具来下载安装别人写的模块。
  - 第三方模块，都是在node核心模块的基础之上，封装了一下，实现了很多非常方便快速简洁的方法。
  - 目前，npm网站收录了超过 160 万个第三方模块。

- **npm的作用**
  - npm的作用是：管理node模块的工具。
    - 下载并安装第三方的模块
    - 卸载第三方模块
    - 发布模块
    - 删除已发布的模块
    - ....

- **本地模块**
  - 初始化
    - 安装本地模块，需要使用npm工具初始化。

```
npm init -y
//或
npm init
//然后一路回车
```

    - 初始化之后，会在项目目录中生成 package.json 的文件。
  - 安装卸载第三方模块的命令
    - 初始化之后，就可以在当前文件夹中安装第三方模块了

```
//建议在安装第三方模块之前，先执行如下命令。
//下面的命令只需要执行一次即可（不管以后重启vscode还是重启电脑，都不需要执行第二次）
npm config set registry https://registry.npm.taobao.org
```

    - 下载安装第三方模块

```
//正常的下载安装
npm install 模块名

//简写install为i
npm i 模块名

//一次性安装多个模块
npm i 模块名 模块名 模块名
```

    - 卸载模块

```
npm uninstall 模块名
npm un 模块名
npm un 模块名 模块名 模块名
```

  - 关于本地模块的说明
    - 下载安装的模块，存放在当前文件夹的 `node_modules` 文件夹中，同时还会生成一个记录下载的文件 `package-lock.json`
    - 下载的模块，在哪里可以使用
      - 在当前文件夹
      - 在当前文件夹的子文件夹
      - 在当前文件夹的子文件夹的子文件夹
      - .....
      - 反过来讲，当查找一个模块的时候，会在当前文件夹的 node_modules 文件夹查找，如果找不到，则去上层文件夹的node_modules文件夹中查找，依次类推。
  - 怎样使用第三方模块
    - 和使用内置模块一样，需要使用 `require` 加载模块
    - 调用模块提供的方法完成工作
    - 不用担心不会用，好的第三方模块都会用使用文档或者官方网站的。
    - 有些模块没有官网，去 github 查找模块的使用文档，或者百度。
  - 演示 moment 模块 的使用
    - http://momentjs.cn/

```
// moment是一个专门处理时间日期的模块
```

```

// 使用模块之前，必须加载
const moment = require('moment');

// 设置语言环境
moment.locale('zh-cn', {
  months: '一月_二月_三月_四月_五月_六月_七月_八月_九月_十月_十一月_十二月'.split('_'),
  monthsShort: '1月_2月_3月_4月_5月_6月_7月_8月_9月_10月_11月_12月'.split('_'),
  weekdays: '星期日_星期一_星期二_星期三_星期四_星期五_星期六'.split('_'),
  weekdaysShort: '周日_周一_周二_周三_周四_周五_周六'.split('_'),
  weekdaysMin: '日_一_二_三_四_五_六'.split('_'),
  longDateFormat: {
    LT: 'HH:mm',
    LTS: 'HH:mm:ss',
    L: 'YYYY-MM-DD',
    LL: 'YYYY年MM月DD日',
    LLL: 'YYYY年MM月DD日Ah点mm分',
    LLLL: 'YYYY年MM月DD日ddddAh点mm分',
    l: 'YYYY-M-D',
    ll: 'YYYY年M月D日',
    lll: 'YYYY年M月D日 HH:mm',
    llll: 'YYYY年M月D日dddd HH:mm'
  },
  meridiemParse: /凌晨|早上|上午|中午|下午|晚上/,
  meridiemHour: function (hour, meridiem) {
    if (hour === 12) {
      hour = 0;
    }
    if (meridiem === '凌晨' || meridiem === '早上' ||
      meridiem === '上午') {
      return hour;
    } else if (meridiem === '下午' || meridiem === '晚上') {
      return hour + 12;
    } else {
      // '中午'
      return hour >= 11 ? hour : hour + 12;
    }
  },
  meridiem: function (hour, minute, isLower) {
    const hm = hour * 100 + minute;
    if (hm < 600) {
      return '凌晨';
```

```
    } else if (hm < 900) {
      return '早上';
    } else if (hm < 1130) {
      return '上午';
    } else if (hm < 1230) {
      return '中午';
    } else if (hm < 1800) {
      return '下午';
    } else {
      return '晚上';
    }
  },
  calendar: {
    sameDay: '[今天]LT',
    nextDay: '[明天]LT',
    nextWeek: '[下]ddddLT',
    lastDay: '[昨天]LT',
    lastWeek: '[上]ddddLT',
    sameElse: 'L'
  },
  dayOfMonthOrdinalParse: /\d{1,2}(日|月|周)/,
  ordinal: function (number, period) {
    switch (period) {
      case 'd':
      case 'D':
      case 'DDD':
        return number + '日';
      case 'M':
        return number + '月';
      case 'w':
      case 'W':
        return number + '周';
      default:
        return number;
    }
  },
  relativeTime: {
    future: '%s内',
    past: '%s前',
    s: '几秒',
    ss: '%d秒',
    m: '1分钟',
```

```
    mm: '%d分钟',
    h: '1小时',
    hh: '%d小时',
    d: '1天',
    dd: '%d天',
    M: '1个月',
    MM: '%d个月',
    y: '1年',
    yy: '%d年'
  },
  week: {
    // GB/T 7408-1994《数据元和交换格式·信息交换·日期和时间表示法》与ISO 8601:1988等效
    dow: 1, // Monday is the first day of the week.
    doy: 4  // The week that contains Jan 4th is the first week of the year.
  }
});

// console.log(moment().format("YYYY-MM-DD HH:mm:ss"))
// console.log(moment().format("L"))
// console.log(moment([2021, 0, 22, 09, 30, 25]).fromNow())
// console.log(moment(13432542333).fromNow())
//console.log(moment('2020-11-10T15:49:05.000Z').fromNow())
```

- 演示jsonwentoken模块
  - jsonwebtoken模块的作用是生成token字符串。
  - https://github.com/auth0/node-jsonwebtoken

```
// 加载模块
const jwt = require('jsonwebtoken');

// console.log(jwt.sign(必填, 必填, 可选, 可选));

// Bearer 不属于token的内容，只是表示token的格式。

// jwt.sign()
// 1. 参数1：对象，要在token保存的数据
// 2. 参数2：加密的字符串，类似于一个钥匙。随便填；后续解密token的时候，需要使用它
// 3. 参数3：对象，配置项，比如配置一下过期时间
// 4. 参数4：生成token后的回调函数

```

```
// console.log('Bearer ' + jwt.sign(
//   { id: 1, username: 'zs' },
//   'shhhhh',
//   { expiresIn: '2h' },
//   // (err, abc) => console.log(abc)
// ));

jwt.sign({ id: 1 }, 'sdfsdf', { expiresIn: '2h' }, (err, result) => {
  if (err) throw err;
  console.log('Bearer ' + result);
});
```

- **package.json文件**
  - 在初始化之后，会生成一个package.json文件
    - 创建 `package.json`

```
npm init
npm init -y
```

    - main
      - main 字段指定了模块的入口文件。
    - dependencies 依赖(复数)
      - dependencies指定了当前项目所依赖的包（就是已经安装了的包的目录，如果直接删除文件目录不会变化，并且使用 `npm install` 可以安装所有的直接删除文件的包）
      - 软件的版本号 jQuery@3.3.1
        - 大版本.次要版本.小版本
        - 小版本：当项目在进行了局部修改或 bug 修正时，修正版本号加 1
        - 次要版本：当项目在原有的基础上增加了部分功能时，主版本号不变，子版本号加 1
        - 大版本：当项目在进行了重大修改或局部修正累积较多，而导致项目整体发生全局变化时，主版本号加 1
      - 版本号前的 `~` 和 `^`
        - 指定版本：比如`1.2.2`，遵循“大版本.次要版本.小版本”的格式规定，安装时只安装指定版本。
        - 波浪号（tilde）+指定版本：比如`~1.2.2`，表示安装1.2.x的最新版本（不低于1.2.2），但是不安装1.3.x，也就是说安装时不改变大版本号和次要版本号。（~1.2.2区间表示：[1.2.2,1.3.0)）
        - 插入号（caret）+指定版本：比如^1.2.2，表示安装1.x.x的最新版本（不低于1.2.2），但是不安装2.x.x。（^1.2.2区间表示：[1.2.2,2.0.0)）
    - scripts

- `scripts`指定了运行脚本命令的 npm 命令行缩写，比如start指定了运行`npm run start`时，所要执行的命令。

```
"scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node app.js",
    "t": "dir c:\\"
 }
```

- 运行 `scripts`

```
npm run t
npm run start
//start可以简化调用，省略“run”
npm start
```

- **require的加载机制**
  - 判断缓存中有没有，如果有，使用缓存中的内容
  - 缓存中没有，那么表示第一次加载，加载完会缓存
  - 判断模块名有没有带路径（./）
  - 模块名中有路径，加载自定义模块（自己写的文件）`const xx = require('./xx')`
    - 优先加载同名文件，加载一个叫做 xx 的文件
    - 再次加载js文件，加载 xx.js 文件
    - 再次加载json文件，加载 xx.json 文件
    - 最后加载node文件，加载 xx.node文件
    - 如果上述文件都没有，则报错 “Cannot find module './xx'”
  - 模块名没有路径，优先加载核心模块，如果没有核心模块，则加载第三方模块
  - 加载第三方模块的查找方式
    - 优先在当前文件夹的node_modules里面查找第三方模块
    - 在当前文件夹的上级目录的node_modules里面查找第三方模块
    - 继续向上层文件夹查找第三方模块
    - .....

- **全局模块**
  - 和本地模块的差异
    - 全局安装的模块，不能通过 `require()` 加载使用。

        - 全局安装的模块，一般都是命令或者工具。
    - 安装卸载命令
        - 安装命令（多一个 `-g`）

```
npm i 模块名 -g
//或
npm i -g 模块名

//mac 系统如果安装不上，使用下面的命令提高权限
sudo npm i -g 模块名
```

        - 卸载命令（也是多一个 `-g`）

```
npm un 模块名 -g
```

        - 全局安装的模块，在系统盘（C盘）
            - 通过命令 `npm root -g` 可以查看全局安装路径
    - 全局安装nodemon模块
        - 安装命令

```
npm i nodemon -g
```

        - nodemon的作用：
            - 代替node命令，启动服务的工具
            - 当更改代码之后，nodemon会自动重启服务。
        - 运行nodemon，如果报错如下：

```
PS D:\96\Node-02\code\http> nodemon .\04-图书管理案例接口.js
nodemon : 无法加载文件 C:\Users\GaoWei\AppData\Roaming\npm\nodemon.ps1，因为在此系统上禁止运行脚本。有关详细
信息，请参阅 https:/go.microsoft.com/fwlink/?LinkID=135170 中的 about_Execution_Policies。
```

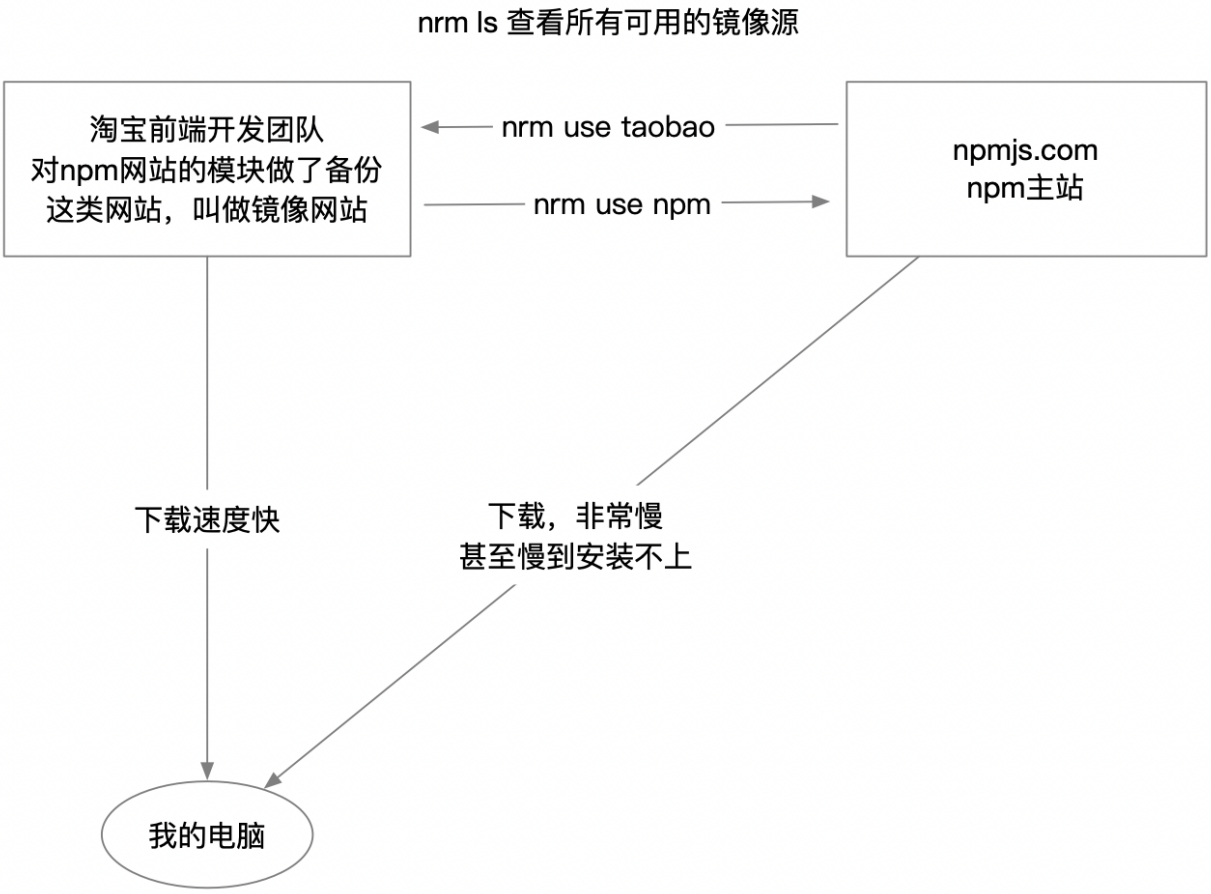
            - 解决办法是：
                - `管理员`方式，打开命令行（powershell）窗口
                - 执行 ` set-ExecutionPolicy RemoteSigned; `
                - 在出现的选项中，输入 `A`，回车。即可
        - 如果报错如下

```
PS C:\Users\97966\Desktop\node练习> node 01-练习.js
nodempn无法将“node”项识别为 cmdlet、函数、脚本文件或可运行程序的名称。请检查
名称的拼写，如果包括路径，请确保路径正确，然后再试一次。
所在位置 行:1 字符: 1
+ node 01-练习.js
+ ~~~~
    + CategoryInfo          : ObjectNotFound: (node:String) [], CommandNotFou
   ndException
    + FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\97966\Desktop\node练习>
```

- 解决办法，重启vscode，win7可能要重启电脑。

- 全局安装nrm模块
  - nrm 是作用是切换镜像源。



  - 全局安装

```
npm i -g nrm    (mac系统前面加 sudo)
```

  - 使用nrm

```
nrm ls    --- 查看全部可用的镜像源
nrm use taobao  ---- 切换到淘宝镜像
```

```
nrm use npm  ---- 切换到npm主站
```

- 运行 nrm 命令会报错：

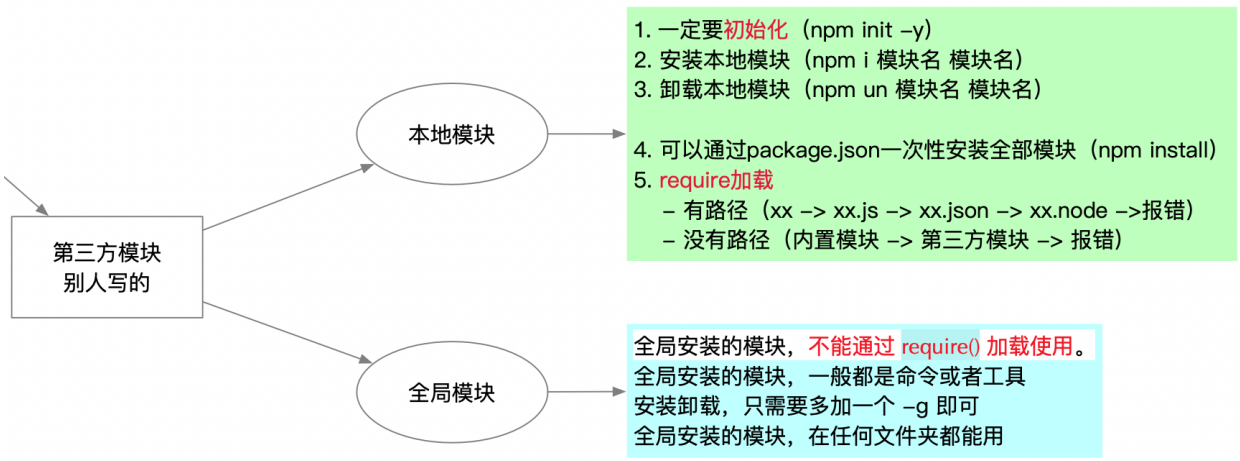
```
PS D:\Lessons\SH\Node02\code> nrm ls
internal/validators.js:124
    throw new ERR_INVALID_ARG_TYPE(name, 'string', value);
    ^

[TypeError [ERR_INVALID_ARG_TYPE]: The "path" argument must be of type string. Received undefined
  at validateString (inte
  at Object.join (path.js  在编辑器中打开文件 (Ctrl + 单击)   按住 Ctrl 键，单击这个文件，就会打开它
  at Object.<anonymous> (C:\Users\LaoTang\AppData\Roaming\npm\node_modules\nrm\cli.js:17:20)
  at Module._compile (internal/modules/cjs/loader.js:1063:30)
  at Object.Module._extensions..js (internal/modules/cjs/loader.js:1092:10)
  at Module.load (internal/modules/cjs/loader.js:928:32)
  at Function.Module._load (internal/modules/cjs/loader.js:769:14)
  at Function.executeUserEntryPoint [as runMain] (internal/modules/run_main.js:72:12)
  at internal/main/run_main_module.js:17:47
] {
  code: 'ERR_INVALID_ARG_TYPE'
}
```

- 将上述标注的文件打开，把17行注释，替换为下面的代码：

```
const NRMRC = path.join(process.env[(process.platform == 'win32') ?
'USERPROFILE' : 'HOME'], '.nrmrc');
```

- **全局模块和本地模块的对比**



- **开发属于自己的包**
  - 规范的包结构
    - 在清楚了包的概念、以及如何下载和使用包之后，接下来，我们深入了解一下包的内部结构。

```
- sy123
        - package.json    (package.json包的配置文件)
        - index.js        (入口文件)
```

```
        📖 - README.md       (说明文档)
```

- 一个规范的包结构，需要符合以下 3 点要求:
  - 包必须以单独的目录而存在
  - 包的顶级目录下要必须包含 package.json 这个包管理配置文件
  - package.json 中必须包含 name，version，main 这三个属性，分别代表包的名字、版本号、包的入口。
    - name 包的名字，我们使用 require()加载模块的时候，使用的就是这个名字
    - version 版本，1.2.18
    - main 入口文件。默认是index.js 。如果不是，需要使用main指定
  - 注意:以上 3 点要求是一个规范的包结构必须遵守的格式，关于更多的约束，可以参考如下网址:
    - https://yarnpkg.com/zh-Hans/docs/package-json
- 开发属于自己的包
  - 初始化 package.json
    - 注意，JSON文件不能有注释，下面加注释，是为了理解。

```json
{
  "name": "sy123",  // 包（模块）的名字，和文件夹同名。别人加载我们的包，找的就是这个文件夹
  "version": "1.0.0",
  "description": "This is a package by Laotang",
  "main": "index.js", // 别人加载我们的模块用，require加载的就是这里指定的文件
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [ // 在npm网站中，通过关键字可以搜索到我们的模块，按情况设置
    "laotang",
    "itcast",
    "test"
  ],
  "author": "Laotang", // 作者
  "license": "ISC" // 开源协议
}
```

  - index.js 中定义功能方法

```
// 别人加载的就是我的 index.js
// 所以，必须在 index.js 中导出内容

```

```
function a() {
  console.log('aaa')
}

function b() {
  console.log('bbb')
}

module.exports = { a, b }
```

- 编写包的说明文档
  - 包根目录中的 README.md 文件，是包的使用说明文档。通过它，我们可以事先把包的使用说明，以 markdown 的 格式写出来，方便用户参考。
  - README 文件中具体写什么内容，没有强制性的要求;只要能够清晰地把包的作用、用法、注意事项等描述清楚即可。

- 注册npm账号
  - 访问 https://www.npmjs.com/ 网站
  - 点击 sign up 按钮，进入注册用户界面
  - 填写账号相关的信息
  - 点击 Create an Account 按钮，注册账号
  - 注册完账号，需要到邮箱中认证一下
- 发布包
  - `终端中`，切换镜像源为npm（不能发布到淘宝，所以必须切换镜像源为npm主站）
  - nrm use npm
  - `终端中`，登录 npm 账号
    - 执行 `npm login` 命令
    - 输入账号
    - 输入密码（输入的密码是看不见的，正常）
    - 输入邮箱

  - 发布
    - 注意，执行命令的文件夹，必须是包的根目录。

    - 运行 `npm publish` 命令，即可将包发布到 npm 上
- 常见错误
    - 自己的模块名（文件夹名）不能和已存在的模块名同名，相似也不行。
    - 没有切换镜像源，会提示如下错误。要发布到npm上，必须切换镜像源为npm

```
npm notice total files:   5
npm notice
npm ERR! code E403
npm ERR! 403 403 Forbidden - PUT https://registry.npm.taobao.org/itheim
a-ucfirst - [no_perms] Private mode enable, only admin can publish this
 module
npm ERR! 403 In most cases, you or one of your dependencies are request
ing
npm ERR! 403 a package version that is forbidden by your security polic
y.

npm ERR! A complete log of this run can be found in:
npm ERR!     /Users/tangfengpo/.npm/_logs/2020-04-07T07_14_35_934Z-debu
```

    - 24小时内不能重复发布

```
npm ERR! code E403
npm ERR! 403 403 Forbidden - PUT https://registry.npmjs.org/itheima-ucf
irst - itheima-ucfirst cannot be republished until 24 hours have passed
.
npm ERR! 403 In most cases, you or one of your dependencies are request
ing
npm ERR! 403 a package version that is forbidden by your security polic
y.

npm ERR! A complete log of this run can be found in:
```

    - 新注册的账号，必须先邮箱（邮件可能是垃圾邮件）验证，然后才能发布

```
npm notice
npm ERR! code E403
npm ERR! 403 403 Forbidden - PUT https://registry.npmjs.org/itheima-abc
 - you must verify your email before publishing a new package: https://
www.npmjs.com/email-edit
npm ERR! 403 In most cases, you or one of your dependencies are request
ing
npm ERR! 403 a package version that is forbidden by your security polic
y.

npm ERR! A complete log of this run can be found in:
npm ERR!     /Users/tangfengpo/.npm/_logs/2020-04-07T07_22_44_119Z-debu
g.log
```

- 删除已发布的包
    - 运行 npm unpublish 包名 --force 命令，即可从 npm 删除已发布的包。
- 注意:
    - npm unpublish 命令只能删除 72 小时以内发布的包
    - npm unpublish 删除的包，在 24 小时内不允许重复发布
    - 发布包的时候要慎重，尽量不要往 npm 上发布没有意义的包